

Page 1: Python Programming

Python is a high-level programming language known for its simple syntax and wide range of applications. It is commonly used for automation, web development, data analysis, machine learning, and artificial intelligence.

Topic	Description	Level
Variables	Store values such as numbers and strings	Beginner
Functions	Reusable blocks of Python code	Beginner
Lists	Ordered collection of values	Beginner
Dictionaries	Key-value data structure	Intermediate

Additional note: This page is included as sample content for testing PDF loading, text extraction, chunking, embeddings, and retrieval in a RAG application. Page number: 1.

Page 2: Java Programming

Java is an object-oriented programming language used for enterprise applications, Android development, backend systems, and many large software projects. Java programs are compiled into bytecode that can run on the Java Virtual Machine.

Concept	Purpose	Example
Class	Defines an object structure	Student
Object	Instance of a class	student1
Inheritance	Reuses parent class features	extends
Encapsulation	Protects internal data	private

Additional note: This page is included as sample content for testing PDF loading, text extraction, chunking, embeddings, and retrieval in a RAG application. Page number: 2.

Page 3: SQL and Databases

SQL is used to store, retrieve, update, and manage data in relational databases. Common database systems include MySQL, PostgreSQL, Oracle Database, and Microsoft SQL Server.

SQL Command	Use	Example
SELECT	Read data	SELECT * FROM users
INSERT	Add data	INSERT INTO users
UPDATE	Modify data	UPDATE users SET name
DELETE	Remove data	DELETE FROM users

Additional note: This page is included as sample content for testing PDF loading, text extraction, chunking, embeddings, and retrieval in a RAG application. Page number: 3.

Page 4: Machine Learning

Machine learning allows computers to learn patterns from data and make predictions or decisions. Supervised learning uses labelled data, while unsupervised learning discovers patterns without labelled outputs.

Algorithm	Type	Typical Use
Linear Regression	Supervised	Prediction
Decision Tree	Supervised	Classification
Random Forest	Supervised	Classification
K-Means	Unsupervised	Clustering

Additional note: This page is included as sample content for testing PDF loading, text extraction, chunking, embeddings, and retrieval in a RAG application. Page number: 4.

Page 5: Artificial Intelligence

Artificial intelligence is a broad field focused on building systems that can perform tasks associated with human intelligence. Modern AI includes natural language processing, computer vision, generative AI, and intelligent agents.

Area	Description	Application
NLP	Works with human language	Chatbots
Computer Vision	Understands images	Image recognition
Generative AI	Creates new content	Text generation
Robotics	Controls intelligent machines	Automation

Additional note: This page is included as sample content for testing PDF loading, text extraction, chunking, embeddings, and retrieval in a RAG application. Page number: 5.

Page 6: LangChain

LangChain is a framework used to build applications around large language models. It provides components for prompts, models, output parsers, retrievers, document loaders, and runnable workflows.

Component	Purpose	Example
Prompt	Structures model input	ChatPromptTemplate
Model	Generates responses	Chat model
Retriever	Finds relevant documents	Vector retriever
Parser	Formats model output	StrOutputParser

Additional note: This page is included as sample content for testing PDF loading, text extraction, chunking, embeddings, and retrieval in a RAG application. Page number: 6.

Page 7: Retrieval-Augmented Generation (RAG)

RAG combines document retrieval with language generation. Documents are loaded, split into chunks, converted into embeddings, stored in a vector database, and retrieved when a user asks a question.

RAG Step	What Happens	Output
Load	Read source documents	Documents
Split	Break documents into chunks	Chunks
Embed	Convert text to vectors	Embeddings
Retrieve	Find relevant chunks	Context

Additional note: This page is included as sample content for testing PDF loading, text extraction, chunking, embeddings, and retrieval in a RAG application. Page number: 7.

Page 8: Embeddings and Vector Stores

Embeddings represent text as numerical vectors. Similar meanings tend to have vectors that are close to each other. Vector stores such as FAISS and Chroma can be used to store and search these vectors efficiently.

Term	Meaning	Purpose
Embedding	Numerical representation of text	Semantic search
Vector	Array of numerical values	Represent meaning
Similarity	Measures closeness	Find related text
Vector Store	Stores searchable vectors	Fast retrieval

Additional note: This page is included as sample content for testing PDF loading, text extraction, chunking, embeddings, and retrieval in a RAG application. Page number: 8.

Page 9: PDF Document Loading

PDF files can be loaded into a RAG pipeline using document loaders. PyPDFLoader is commonly used with LangChain to extract text from PDF pages. Each page can become a document object containing page content and metadata.

Item	Description	Example
PDF	Portable document file	research.pdf
Loader	Reads the PDF	PyPDFLoader
Page Content	Extracted text	Paragraphs
Metadata	Extra document information	Page number

Additional note: This page is included as sample content for testing PDF loading, text extraction, chunking, embeddings, and retrieval in a RAG application. Page number: 9.

Page 10: RAG Question Answering

A typical RAG question-answering workflow receives a user question, retrieves relevant document chunks, places those chunks into a prompt, and sends the prompt to a language model. The model then produces an answer based on the retrieved context.

Stage	Input	Output
Question	User query	Question text
Retrieval	Question + vector store	Relevant documents
Prompt	Question + context	Model input
Generation	Prompt	Final answer

Additional note: This page is included as sample content for testing PDF loading, text extraction, chunking, embeddings, and retrieval in a RAG application. Page number: 10.